

How it works

This page follows one leaked key, from the moment it is found to the report you read.

The example

Riya works on DemoPay, a small payments app. She pastes a live Stripe key into a one-off script, `scripts/migrate_customers.py`. A Stripe key is the password that lets an app take real card payments.

She **commits** the script. A commit is a saved snapshot of the project in **Git**, the tool that keeps a project's history.

A day later she deletes the script and commits again. The key is gone from the latest code. It is not gone from Git.

Step 1: Find

SecureGate asks **Gitleaks**, a free scanner, to read every commit. Gitleaks knows the shapes of many kinds of keys. Stripe live keys start with `sk_live_`, so Gitleaks spots this one in the commit where Riya added it.

At this moment the full key exists only in SecureGate's memory. It is never printed or saved.

With `--scanners all`, a second scanner, **TruffleHog**, reads the history too. It can also ask Stripe whether the key still works, which Gitleaks cannot. When both find the same key on the same line, SecureGate reports it once and names both.

Step 2: Decide

SecureGate checks the finding against the rules in `policy.yaml`, from top to bottom. Each rule has its number from SecureGate's policy table. The first rule that matches decides:

1. Rule 1, `verified-live`: did the provider confirm that the key works right now? Only TruffleHog can ask; here nobody did.
2. Rule 3, `placeholders`: is it an obvious fake, like `changeme`? No.
3. Rule 7, `tests-fixtures-docs`: is it in tests, fixtures or docs, or in a Markdown or example file? No.
4. Rule 8, `provider-keys`: does the value have the format of a live payment, cloud or GitHub key, or is it a private key? It starts with `sk_live_`: **yes, so: block.**

The rules after it, for test-mode keys, passwords written in the code, risky handling of secrets and everything else, are never reached.

Step 3: Report

SecureGate hides the key before it shows anything. This is called **masking**. A value of 16 or more characters keeps only its first 4 and last 4 characters; a shorter one becomes `****`.

It also makes a **fingerprint**: a code that recognizes the same key again later, without storing the key.

DECISION	RULE	FILE:LINE	VALUE
block	stripe-access-token	scripts/migrate_customers.py:3	sk_l****562d

Blocked:

scripts/migrate_customers.py:3 sk_l****562d

why: provider-keys: A payment, cloud or private key gives direct access to money, data or servers.

fix: Treat it as leaked. Rotate (replace) the key at the provider now, remove it from the code, and load it from an environment variable or a secrets manager instead.

For every blocked finding it says why it was blocked and how to fix it. Both lines come from the matching rule in `policy.yaml`: its `reason` and its `remediation`.

The full details go to `findings.json`. SecureGate then ends with **exit code 1**. An exit code is the number a program gives back when it finishes, so that other tools can react. 1 means "at least one finding is blocked".

Why deleting the line is not enough

Git keeps every commit. Anyone with a copy of the project can open the old commit and read the key. Deleting the line only hides it from the newest version.

The only real fix is **rotation**: create a new key at Stripe, switch the app to it, and cancel the old one. The leaked key then stops working.

What block, warn and ignore mean

Decision	What it means	What to do
block	A real secret is very likely exposed. SecureGate ends with exit code 1, which stops the commit on your laptop and turns the pull request's check red on GitHub.	Rotate the secret, then remove it from the code.
warn	Maybe a secret, maybe not. It is reported, but it does not stop anything.	Check it. If it is real, treat it like a block.
ignore	Not a secret, for example the placeholder <code>YOUR_API_KEY_HERE</code> . It is only listed in <code>findings.json</code> .	Nothing.

The whole journey

